Лекція №16 (2 години). Хуки стану: useState та керовані компоненти.

План лекції

1. Поняття Стану (State) у React та чим він відрізняється від звичайних змінних.
2. Введення в Хуки (Hooks) та правила їх використання.
3. Робота з `useState`: синтаксис та ініціалізація.
4. Асинхронна природа оновлення стану.
5. Робота зі складним станом (об'єкти та масиви) та їхня немутабельність.
6. Керовані компоненти (Controlled Components): зв'язування стану з формами.
7. Підняття стану (Lifting State Up).

Перелік умовних скорочень

- **UI** (User Interface) — користувацький інтерфейс.
- **Hook** — хук (гачок), спеціальна функція React для підключення до можливостей бібліотеки.
- **State** — стан (пам'ять) компонента.
- **DOM** (Document Object Model) — об'єктна модель документа.

Вступ

Досі ми розглядали компоненти просто як функції, які отримують дані ззовні (Props) і малюють HTML. Але більшість компонентів у реальних додатках мають бути інтерактивними. Світла чи темна тема, товари в кошику, текст, який користувач зараз вводить у поле пошуку — все це дані, які постійно змінюються з часом.

Якби ми зберігали ці дані у звичайних змінних (`let counter = 0`), React би просто проігнорував їхні зміни. Інтерфейс (UI) не знав би, що `counter` став рівним 1, і на екрані залишився б нуль. Щоб змусити React "перемалювати" компонент при зміні даних, ми повинні використовувати **Стан (State)**.

З 2018 року основним способом управління станом у функціональних компонентах є технологія **Hooks** (Хуки). Мета цієї лекції — оволодіти найголовнішим хуком — `useState`, зрозуміти концепцію незмінності (немутабельності) та навчитися працювати з HTML-формами у стилі React.

1. Поняття Стану (State)

Стан (State) — це локальна "Пам'ять" компонента. Це дані, які:

1. Визначають, як зараз виглядає компонент та як він поводиться.
2. Можуть змінюватися з часом (внаслідок дії користувача, відповіді від сервера тощо).
3. **Головне:** Коли Стан змінюється, React АВТОМАТИЧНО запускає функцію компонента наново, перераховує новий JSX і оновлює сторінку (запускає Реконсиляцію).

> ***Різниця між Props і State:*** *Props передаються ззовні (як аргументи функції), і їх не можна змінювати. State створюється та зберігається всередині самого компонента, і це єдині дані, які компонент має повне право змінювати самостійно.*

2. Введення в Хуки (Hooks) та їх правила

Хук (від англ. hook — гачок) — це спеціальна вбудована функція React, яка дозволяє функціональним компонентам "підчепитися" до певних фіч React (наприклад, до управління станом або життєвим циклом), які раніше були доступні тільки класовим компонентам. Всі хуки традиційно починаються зі слова `use`.

Два залізні правила ̄ків (Rules of Hooks):

1. **Викликати тільки на верхньому рівні (Top Level):** Не можна викликати хуки всередині циклів `for`, умов `if` або вкладених функцій. З погляду рушія React, порядок виклику хуків ніколи не повинен змінюватися від рендеру до рендеру.
2. **Викликати тільки всередині React-функцій:** Хуки можна використовувати тільки всередині функціональних компонентів або всередині власних кастомних хуків.

3. Робота з `useState`

`useState` — це фундаментальний хук. Він використовується для оголошення змінної стану.

Щоб його використати, спочатку його треба імпортувати:

```
// Імпортуємо вгорі файлу
import { useState } from "react";
```

Синтаксис (`useState` завжди використовує ручну деструктуризацію масиву):

```
// Початкове значення - 0
```

```
const [count, setCount] = useState(0);
```

Розберемо цей рядок:

1. `useState(0)` — ми кажемо: "Дай мені стан, який зі старту дорівнює 0".
2. `useState` повертає масив з рівно двома елементами.
3. `count` (перший елемент) — це константа, в якій лежить ПОТОЧНЕ значення (зараз 0).
4. `setCount` (другий елемент) — це спеціальна ФУНКЦІЯ (сеттер). ЄДИНИЙ законний спосіб змінити `count` — це викликати `setCount(нове_значення)`.

Приклад (Лічильник натискань):

```
import { useState } from "react";

function CounterButton() {
  const [clicks, setClicks] = useState(0);

  const handleClick = () => {
    // Кажемо React: зміни clicks на (clicks + 1)
    // React побачить зміну і перемалює цей компонент автоматично
    setClicks(clicks + 1);
  };

  return <button onClick={handleClick}>Ти натиснув мене {clicks} разів</button>;
}
```

Якщо ви спробуєте написати `clicks = clicks + 1;` — змінна в пам'яті оновиться, але на екрані НІЧОГО не зміниться, бо React не дізнається про цю махінацію без виклику `setClicks()`.

4. Асинхронна природа оновлення стану

Одна з найпоширеніших пасток для новачків — нерозуміння того, як саме працюють сеттери (наприклад, `setClicks`).

```
const handleClick = () => {
  setClicks(clicks + 1);
  // Що виведе консоль одразу після зміни?
  console.log(clicks);
};
```

Консоль виведе СТАРЕ значення! Функції зміни стану в React (сеттери) працюють **асинхронно**. Вони лише створюють "заявку" на оновлення компонента, але не зупиняють виконання поточного коду і не змінюють константу `clicks` миттєво. Резервування нового значення відбудеться тільки в **наступному рендері** компонента.

Якщо вам потрібно двічі підряд оновити стан в одному кліку на основі попереднього, потрібно передавати у сеттер не значення, а **callback-функцію**:

```
const handleDoubleIncrease = () => {
  // React гарантує, що в prev (previous) буде найсвіжіше значення з черги
  setClicks((prev) => prev + 1);
  setClicks((prev) => prev + 1);
};
```

5. Складний стан та Немутабельність

Коли ви зберігаєте об'єкти чи масиви у стані, ви повинні пам'ятати про концепт **Немутабельності (Immutability)** — тобто незмінності.

В React **суворо заборонено** змінювати (мутувати) старий об'єкт або масив стану напряму.

- Погано: `userState.age = 25; setUserState(userState);`
- Погано: `ordersList.push(newOrder); setOrders(ordersList);`

Чому? Тому що React використовує не глибоке порівняння, а порівняння за "посиланням у пам'яті" (Reference equality). Якщо ви змінили стару коробочку `ordersList.push` і передали її ж у `setOrders`, React подумає: *"Ага, посилання на об'єкт те ж саме, отже нічого не змінилося. Я не буду перемальовувати сайт"*.

Правило: Завжди СТВОРЮЙТЕ НОВИЙ об'єкт чи масив (копію старого), змінюйте копію і передавайте її в сеттер! Найпростіше це робити за допомогою `spread оператора ...` (Трьох крапок).

Робота з Масивом:

```
const [todos, setTodos] = useState(["Купити хліб"]);

const addTodo = () => {
  // Створюємо НОВИЙ масив [], кладемо туди все зі старого (...todos), і додаємо
нове.
  setTodos([...todos, "Зробити домашку"]);
};
```

Робота з Об'єктом:

```
const [user, setUser] = useState({ name: "Іван", age: 20 });

const updateAge = () => {
  // Створюємо НОВИЙ об'єкт {}, копіюємо всі поля (...user), а поле 'age' -
ПЕРЕЗАПИСУЄМО
  setUser({ ...user, age: 21 });
};
```

6. Керовані компоненти (Форми у React)

У звичайному HTML, `<input>` сам тримає власний текст всередині себе. У React всі дані мають бути централізовані в React State. Це створює концепцію **Керованого Компонента (Controlled Component)**.

Щоб поле стало керованим:

1. Його атрибут `value` жорстко прив'язується до змінної зі стану.
2. Його подія `onChange` викликає сеттер і оновлює стан.

```
function RegistrationForm() {
  // Стан для інпуту
  const [login, setLogin] = useState("");

  const handleChange = (e) => {
    // e.target.value — це те, що користувач щойно надрукував на клавіатурі
    setLogin(e.target.value);
  };

  return (
    <form>
      <label>Ваш логін: </label>
      <input type="text" value={login} onChange={handleChange} />
      {/* Текст параграфа миттєво змінюватиметься з кожним натисканням клавіші */}
      <p>Довжина вашого логіна: {login.length} символів.</p>
    </form>
  );
}
```

Якщо ви вкажете `<input value={login} />`, але забудете дописати `onChange`, поле стане **заблокованим**. Користувач не зможе нічого туди надрукувати, бо React жорстко триматиме значення `login` (яке дорівнює порожньому рядку).

7. Підняття стану (Lifting State Up)

Де має жити (оголошуватися) стан? State завжди належить лише одному конкретному компоненту (де був викликаний `useState`). Пропси можуть їхати **ТІЛЬКИ ВНИЗ**, до дітей. Але що робити, якщо два різні компоненти (наприклад, `<Sidebar>` і `<MainContent>`) на одному рівні потребують доступу до однієї і тієї ж інформації (наприклад, відкрито чи закрито меню)?

Рішення: Ми переносимо (піднімаємо) цей загальний стан вгору — у їхнього найближчого спільного компонента-Батька (наприклад, в `App.jsx`).

Батько зберігає стан `isOpen` і відправляє його значення в `<MainContent>` через Props. Як дозволити `<Sidebar>` закрити меню? Батько передає в `<Sidebar>` через Props не тільки значення, але й **функцію-сеттер**!

```
function App() {
  const [isOpen, setIsOpen] = useState(false);

  // Батько спускає дітям пропси та функції
  return (
    <div>
      {/* Меню отримує функцію для зміни стану свого батька! */}
      <Menu isOpen={isOpen} closeMenu={() => setIsOpen(false)} />

      <MainContent showingPanel={isOpen} />
    </div>
  );
}
```

Висновки

1. **State (Стан)** — це механізм зберігання даних всередині компонента. Зміна стану гарантовано викликає перерендеринг компонента і оновлення інтерфейсу.
2. Хуки (`hooks`) — це функції (на кшталт `useState`), що дозволяють підключатися до екосистеми React. Вони повинні викликатися виключно на верхньому рівні функціонального компонента.

3. Хук `useState` повертає масив із поточної константи-значення та функції для її зміни (яка працює асинхронно).
4. Змінювати (мутувати) об'єкти і масиви з поточного стану напряму — заборонено. Потрібно створювати їхні незалежні копії (наприклад, через spread-оператор `...`), змінювати їх, і передавати у State Setter.
5. Робота з формами в React будується за патерном Керованих Компонентів (Controlled Components), коли кожен натиск клавіші оновлює React State через подію `onChange`, а атрибут `value` прив'язано до цього стану.
6. Якщо стан потрібен відразу кільком компонентам на одному рівні дерева, застосовується підняття стану (Lifting State Up) до їхнього найближчого спільного предка.

Джерела

1. React Docs: State: A Component's Memory
2. React Docs: Updating Objects in State — докладно про немутабельність та spread `...` оператор.
3. React Docs: Sharing State Between Components — підняття стану.
4. React Docs: Rules of Hooks — Чому хуки не можна обгортати в if.

Запитання для самоперевірки

1. Чим `State` фундаментально відрізняється від `Props` з точки зору права на зміну даних?
2. Що станеться, якщо ви спробуєте оновити значення лічильника напряму через математичний оператор (наприклад, `count = count + 1`), не використовуючи функцію `setCount()`?
3. Чому виклики хуків (усіх функцій зі словом `use`) заборонено ставити всередину умовних конструкцій `if () {...}`?
4. Що означає термін "асинхронне оновлення стану"? Яке значення побачить console.log(X) одразу після рядка з виконання `setX(...)`?
5. Чому React "категорично забороняє" використовувати вбудований метод `push()` для масивів, які зберігаються у стейті? Як правильно додати елемент?
6. Що таке Керований Компонент (Controlled Component) на прикладі поля вводу `<input />`? Які два атрибути (пропси) обов'язкові для його правильної роботи?
7. Поясніть логіку "Підняття стану". Якщо брат і сестра компоненти потребують одних даних, де варто оголосити змінну через `useState`?